

Client Invoicing API

Project Design & Specification

Alex Khouri - 2026

Resource Model.....2

Custom Types.....2

Data Model3

Data Controllers.....4

Endpoint Definitions.....5

Authentication/Authorisation6

Approval Workflow6

Working Assumptions.....6

Resource Model

Custom Types

- INVOICE_ACTION
 - o NULL
 - o REPORT
 - o CREATE
 - o DELETE
 - o ADJUST
 - o DRAFT
 - o APPROVE
 - o SEND
 - o PAY
- INVOICE_ENTRY
 - o NULL
 - o ADD
 - o REMOVE
 - *NB: Invoice entry removal is not implemented at this time. Therefore, reverting individual entries is performed by adding corrections as new entries.*
- INVOICE_STATE
 - o NULL
 - o DRAFT
 - o APPROVED
 - o SENT
 - o PAID
- USER_ROLE
 - o NULL
 - o STAFF
 - o MANAGER

Data Model

NB: Object references prefixed by “parent” are used to implement 2-way relationships within the data model. These are implemented using the ‘any’ data type within the source code (to prevent circular imports when lower-hierarchy classes import upper-hierarchy classes). However, their data types are specified here based on ideological system architecture.

Client

Name	string
Projects	dictionary[string, Project]

Invoice

ID	integer
Entries	list[InvoiceEntry]
Total	float
State	INVOICE_STATE
ParentProject	Project

InvoiceAdjustment

Entry	InvoiceEntry
Type	INVOICE_ENTRY
User	User
ParentInvoice	Invoice

InvoiceEntry

Date	date
Amount	float
Description	string
ParentInvoice	Invoice

Project

Name	string
Invoices	dictionary[integer, Invoice]
ParentClient	Client

User

Username	string
Password	string
Role	USER_ROLE

Data Controllers

ClientProjectController

- Manages project data for each client

Clients	dictionary[string, Client] <i>Client Name >> Client</i>
Projects	dictionary[string, [dictionary[string, Project]] <i>Client Name >> Project Name >> Project</i>
InvoiceControllers	dictionary[Project, InvoiceController]

InvoiceController

- Manages invoice data for each client project

Invoices	dictionary[integer, Invoice] <i>Invoice ID >> Invoice</i>
InvoiceAdjustments	dictionary[Invoice, list[InvoiceAdjustment]]
NextInvoiceID	integer
ParentProject	Project

UserAccessController

- Manages user login sessions
- Manages system access restrictions for user roles

Users	dictionary[string, User] <i>Username >> User</i>
SessionTokens	dictionary[integer, User] <i>Session Token >> User</i>
NextSessionToken	integer

Endpoint Definitions

Endpoint	Message Type(s)	URL Argument(s)	System Action(s)
/register	POST	• username(string) • password (string) • roleString (string)	• User registration
/login	GET	• username(string) • password (string)	• User login
/logout-session	GET	• session (integer)	• User logout (single session)
/logout-all	GET	• session (integer)	• User logout (all sessions)
/client/<clientName>	GET	• session (integer) • outstandingOnly (string: "true", "false")	• Get client invoice report • Yield outstanding invoices only via argument
/client/<clientName> /project/<projectName>	GET	• session (integer) • outstandingOnly (string)	• Get project invoice report • Yield outstanding invoices only via argument
/client/<clientName> /project/<projectName> /invoice	POST	• session (integer)	• Create invoice
/client/<clientName> /project/<projectName> /<invoiceID>	GET	• session (integer)	• Get individual invoice report
/client/<clientName> /project/<projectName> /<invoiceID>	DELETE	• session (integer)	• Delete invoice
/client/<clientName> /project/<projectName> /<invoiceID>	PATCH	• session (integer) • action (INVOICE_ACTION) - o ADJUST - o DRAFT - o APPROVE - o SENT - o PAY <i>NB: Other actions are enabled by other HTTP message types</i> • amount (float) • description (string)	• Adjust invoice • Draft invoice • Approve invoice • Pay invoice

Authentication/Authorisation

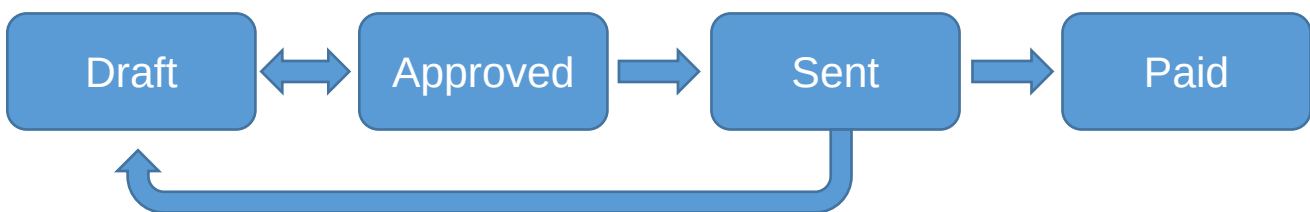
Authentication is provided by submitting username & password credentials, which results in a session token being provided to the caller if their credentials are valid. Storing session tokens inside a hashable data structure allows sessions to be looked up quickly, reducing latency during both authentication and authorisation checks. By using the session tokens as keys for this collection (rather than User object references), multiple simultaneous login sessions can be supported for each user (thereby enabling access from multiple devices).

Once users are authenticated, they're required to provide their session token for any HTTP request that requires authorisation. The tokens are used to identify the sender of each request, and then determine their role within the organisation (which determines which system actions they're authorised to perform).

All session-related functionality is managed by the Session Controller, which manages session tokens and the central register of all valid users. It also implements the access restriction logic for user roles.

Approval Workflow

This has been defined using the following state transition structure:



Only managers can approve invoices. All other invoice state transitions are performed by staff.

Working Assumptions

- Invoices **must** be re-approved after any adjustments are made.
 - **Reason:** Failure to follow this process could allow staff to make unapproved changes to invoices (prior to sending them to clients).
- Invoices **can** be marked as drafts after they're sent (due to the need for further edits).
 - **Reason:** Clients may rightfully dispute entries on invoices, requiring staff to draft new versions of the invoice prior to sending an updated version.
- If invoices need to be edited after being sent, staff are responsible for updating their state.
 - **Reason:** Staff are responsible for creating invoices, which involves setting their state to 'Draft' (which is the same state that would be used when they're returned by clients).
- Invoices **can't** be edited once paid
 - **Reason:** The act of paying an invoice infers agreement that its details require no further updates. Retrospectively editing paid invoices is also a form of fraud.
- Invoices can be deleted after being created (with deletion managed by staff).
 - **Reason:** Invoices may be created for transactions or projects that are cancelled.
- Staff can only obtain data for individual invoices, while managers can obtain data for individual invoices **and** all invoices filed under clients or projects.
 - **Reason:** This provides staff with ability to check invoices before making adjustments to them, while providing managers with high-level reporting capability.